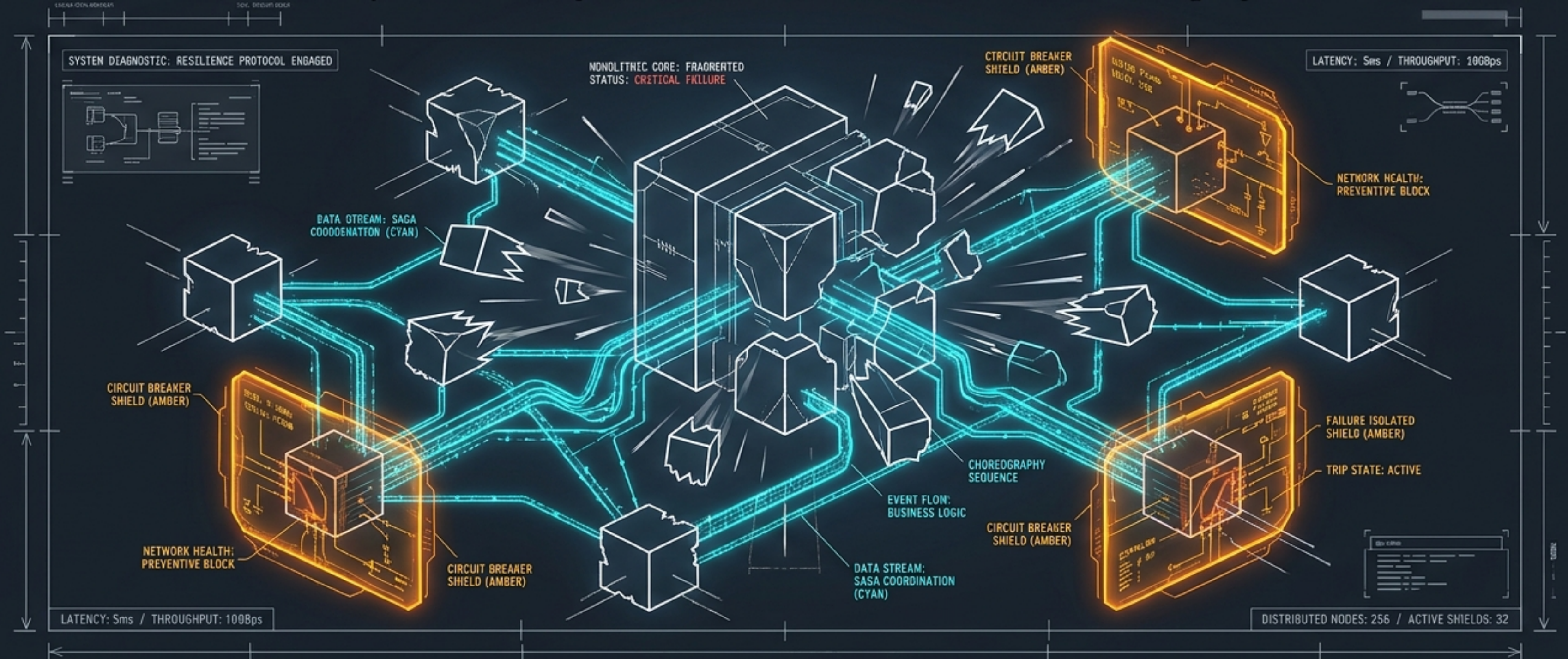


Resiliencia Distribuida

Resolviendo el colapso de red y la consistencia de datos con Saga y Circuit Breaker



LA DUALIDAD DEL PROBLEMA EN LOS MICROSERVICIOS

STATE MACHINE: DATA FLOW

El Desafío de los Datos (Lógica de Negocio)

AXIOMA

El patrón Base de Datos por Servicio rompe las transacciones ACID tradicionales.



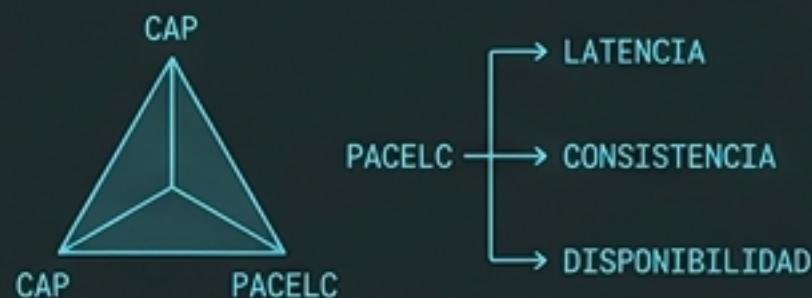
PROBLEMA

Forzar bloqueos distribuidos (Two-Phase Commit) genera cuellos de botella sistémicos.



TEOREMA

CAP y PACELC nos obligan a elegir entre latencia, consistencia y disponibilidad.



CIRCUIT SCHEMATIC: INFRASTRUCTURE

El Desafío de la Red (Infraestructura)

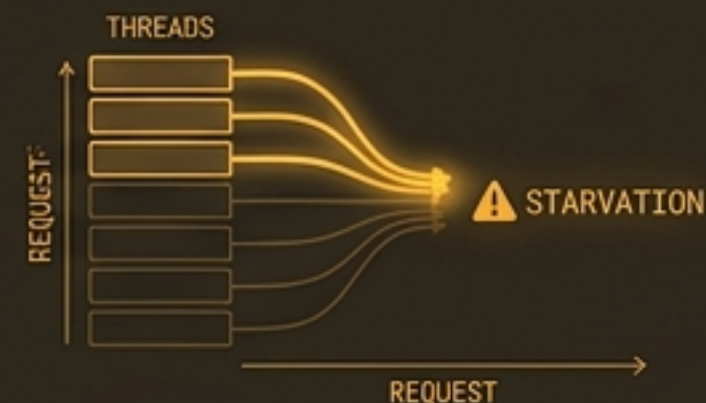
AXIOMA

La red es inherentemente no confiable.



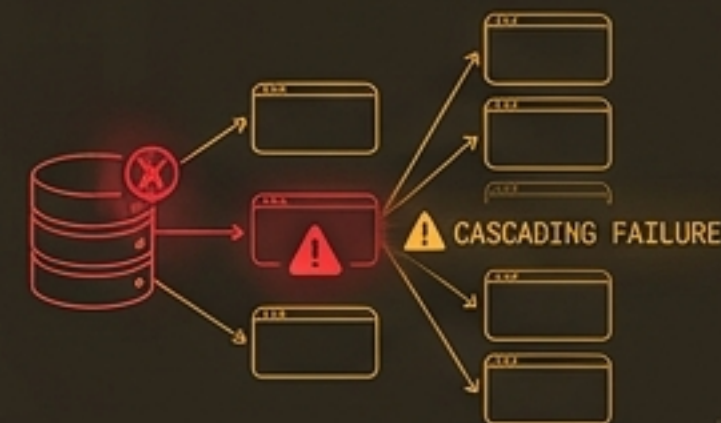
PROBLEMA

La comunicación sincrónica entre servicios provoca inanición de hilos (thread starvation).



CONSECUENCIA

Un fallo aislado en una base de datos desencadena un colapso masivo en cascada.



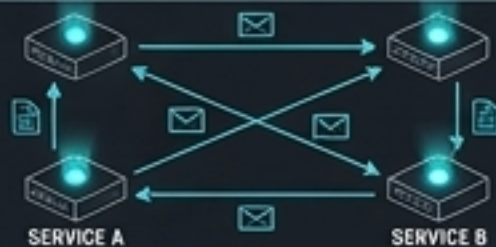
DOS DIMENSIONES OPERATIVAS ORTOGONALES

No son patrones competidores; abordan patologías sistémicas radicalmente distintas.



ANATOMÍA DE SAGA: RECONCILIANDO LA CONSISTENCIA

Descomponiendo una transacción global en transacciones locales aisladas.

	Saga por Coreografía 	Saga por Orquestación 
Naturaleza	Descentralizada (Basada en Eventos) 	Centralizada (Comando y Control) 
Acoplamiento	Extremadamente bajo (Servicios autónomos) 	Moderado a alto (Orquestador conoce todo) 
Observabilidad	Fragmentada (Requiere rastreo distribuido) 	Centralizada (Estado exacto en una base de datos) 
Escalabilidad	Sobresaliente (Sin cuellos de botella) 	Limitada por la capacidad del orquestador 



DECISIÓN ARQUITECTÓNICA



RabbitMQ

Para plataformas de comercio electrónico ágiles, la Coreografía maximiza el rendimiento global, eliminando el bloqueo de bases de datos mediante la comunicación sobre RabbitMQ.

El flujo nominal: **Consistencia eventual** en acción

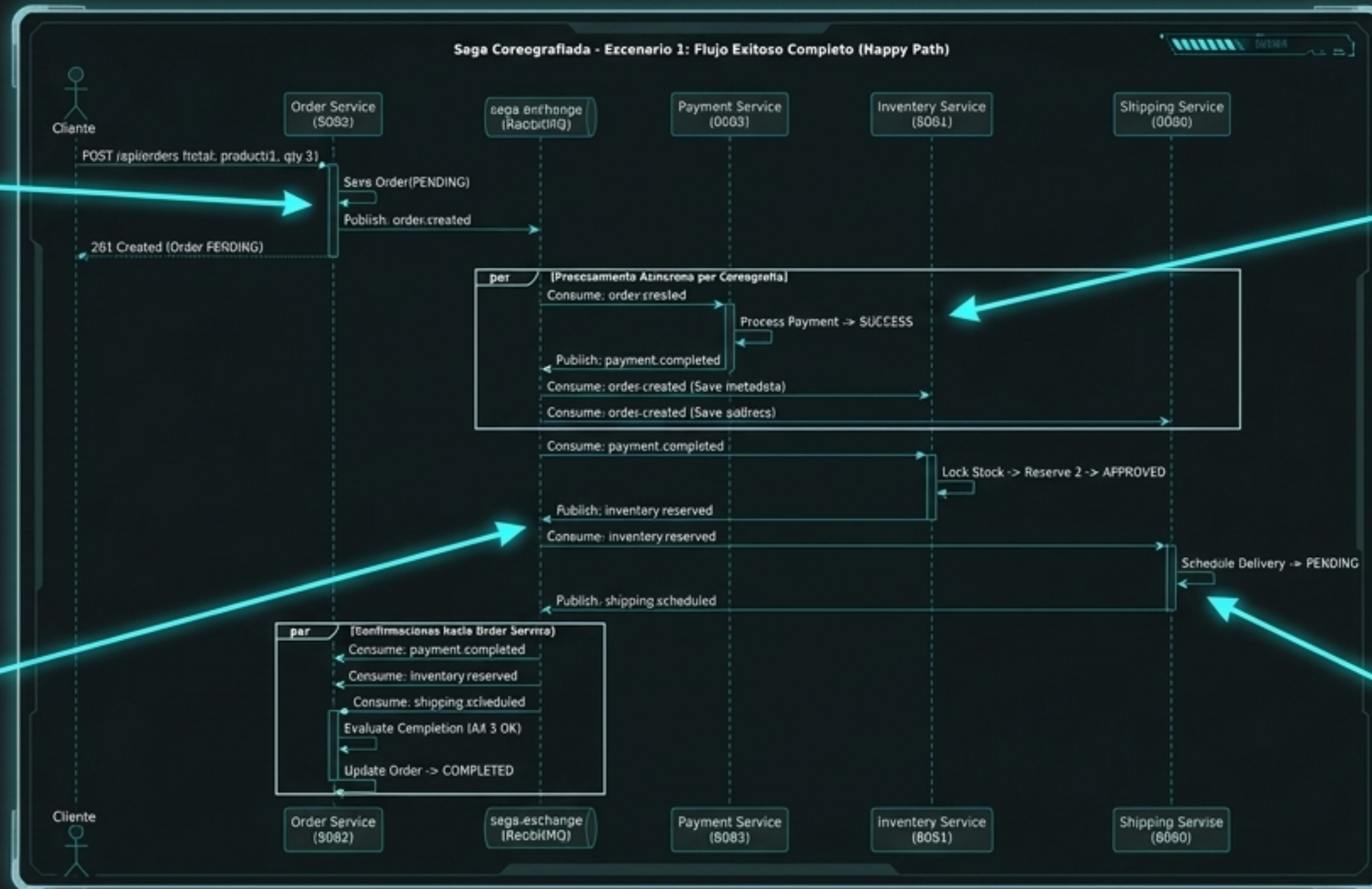
Escenario 1 - Éxito de extremo a extremo sin bloqueos centrales.

1. Creación: Order Service guarda orden (PENDING) y emite `order.created`.

3. Reserva: Inventory Service escucha el cobro, aplica bloqueo pesimista a su tabla y emite `inventory.reserved`.

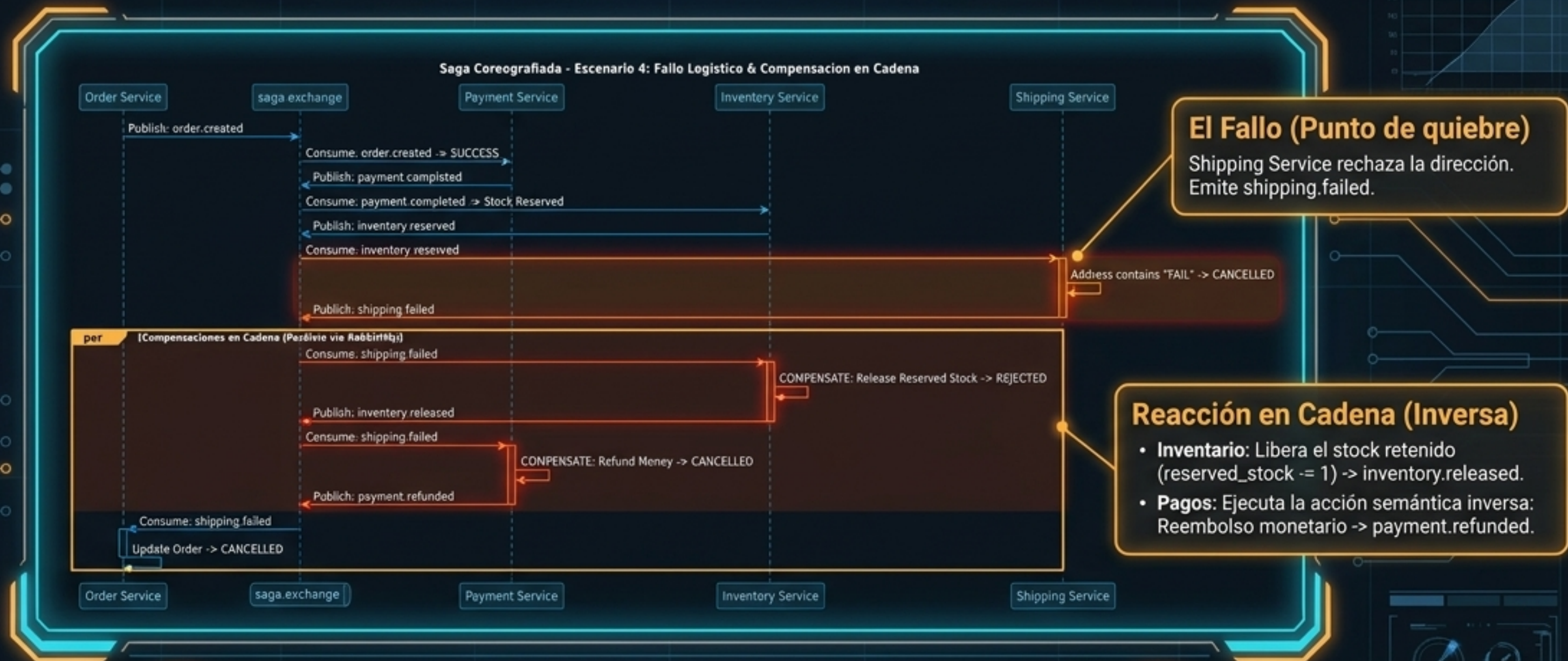
2. Cobro: Payment Service reacciona, descuenta fondos de forma local y emite `payment.completed`.

4. Despacho y Cierre: Shipping Service programa logística. Order Service recopila los eventos exitosos y muta el estado a **COMPLETED**.



La ruta de fallo: Transacciones Compensatorias

Escenario 4 - Cuando el Rollback tradicional es matemáticamente imposible.



Innegociable (Regla de Negocio)

Toda compensación debe ser implacablemente reintentada y estrictamente idempotente para evitar dobles reembolsos ante duplicación de eventos.

La amenaza de la Infraestructura: El Fallo en Cascada

¿Por qué Saga no es suficiente para problemas de red?

1. El Origen

RESPONSE_TIME: 30000ms (TIMEOUT)



30s

STATUS: DEGRADED

LATENCY_ALERT

Una dependencia remota (ej. Inventory Service) sufre degradación grave y tarda 30 segundos en responder.

2. La Inanición (Starvation)

THREAD_POOL: 50/50 (FULL)



100%
EXHAUSTED

TCP_CONNECTIONS: MAX

WAITING_FOR_RESPONSE

El servicio consumidor agota todo su pool de hilos HTTP (conexiones TCP) esperando la respuesta que nunca llega.

3. El Colapso Sistémico

SYSTEM_STATUS: CRITICAL FAILURE



CASCADING_EFFECT: ACTIVE

DISTRIBUTED_LOCK: DETECTED

Otros servicios que dependen del consumidor comienzan a atascarse. El problema aislado inmoló todo el ecosistema distribuido.



Insight: Necesitamos un mecanismo proxy de protección.
Necesitamos un Disyuntor de Red.

El Sistema Inmunológico de la Red

Máquina de estados finita del Circuit Breaker (Resilience4j)



PRESSURE 303
PROJECT 200 PM
PRESSURE 238

CLOSED

CLOSED (Operación Normal)

El flujo fluye libremente. El sistema monitorea silenciosamente la ventana estadística de fallos y latencia.

OPEN

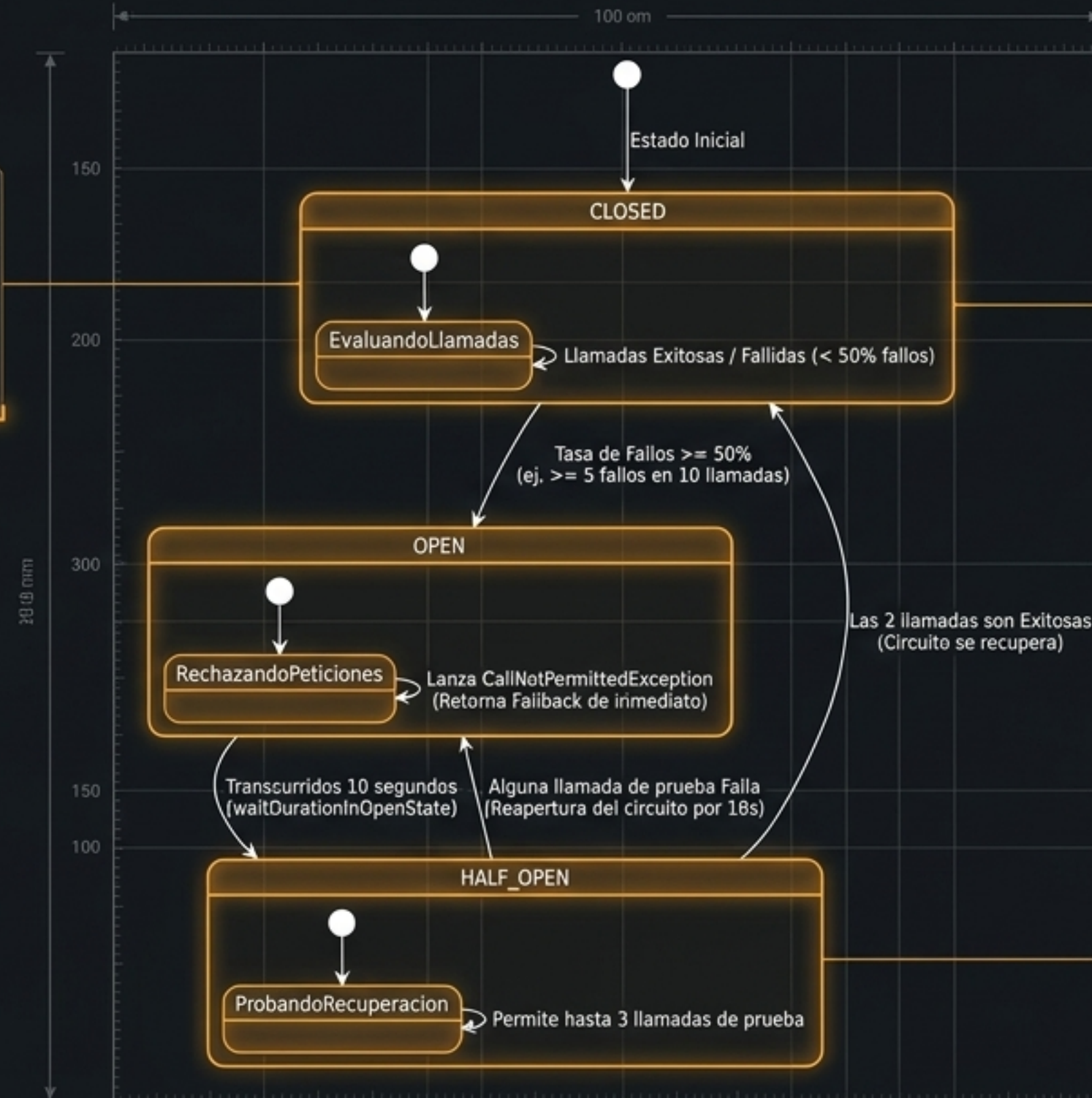
OPEN (Rechazo Preventivo)

Umbral superado. El circuito se rompe. Las peticiones abortan en nanosegundos localmente (Fail-Fast) sin consumir hilos de red, otorgando un respiro al proveedor agonizante.

HALF-OPEN

HALF-OPEN (Sondeo Exploratorio)

Transcurrido un tiempo de espera, se permite un volumen raquítico de tráfico. Si tiene éxito, el circuito sana (CLOSED); si falla, recae (OPEN).



Pressure



Temp

Short-Circuiting y Degradación Elegante

Intercepción y contingencia en tiempo real.

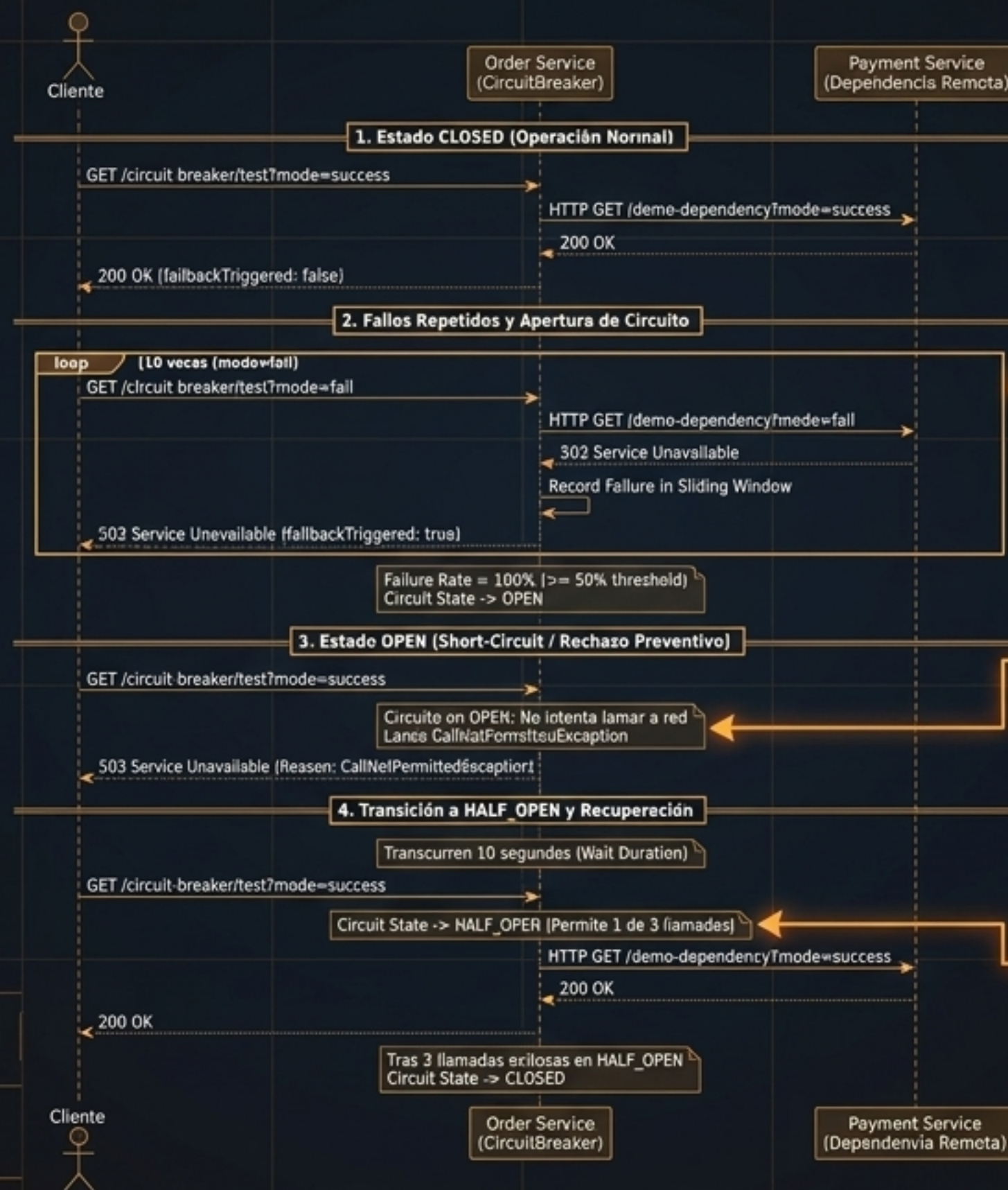
RESPONSE_TIME: 30000ms (TIMEOUT)



30s

STATUS: DEGRADED

TEP_CONNECTIONS: MAX



Ventana de Fallos: Tras repetidos 503 Service Unavailable, la tasa alcanza el 100%. El circuito se dispara.

Fail-Fast (Rechazo instantáneo): En estado OPEN, la red ni siquiera se toca. Se lanza `CallNotPermittedException` de inmediato.

Fallback (Contingencia): En lugar de arrojar un error crudo al cliente, el sistema provee gracia: entrega un valor de caché, encola la petición o devuelve un mensaje contextual.

Calibración de Signos Vitales

El delicado equilibrio matemático para evitar flapping estéril en la red.

`slidingWindowSize = 10`



Dictamina el volumen del historial en el búfer circular. Ventanas inmensas hacen al circuito letárgico; ventanas pequeñas causan alta volatilidad.

`failureRateThreshold = 50%`



El porcentaje estadístico de anomalías (ej. 5 de 10 llamadas) requerido para disparar el estado OPEN.

`waitDurationInOpenState = 10s`



El periodo de aislamiento ininterrumpido. Debe coincidir con el tiempo típico de reinicio de un pod o purga de colas.

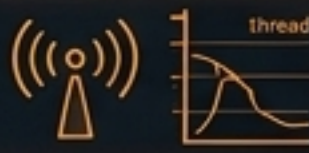
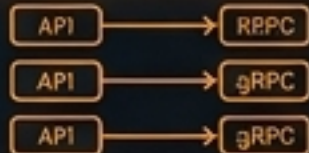
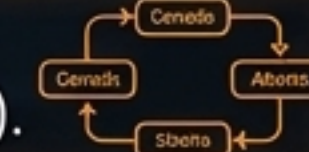
`permittedNumberOfCalls = 3`



Las sondas en estado HALF_OPEN. Evita transicionar falsamente al éxito por un simple golpe de suerte estadístico.

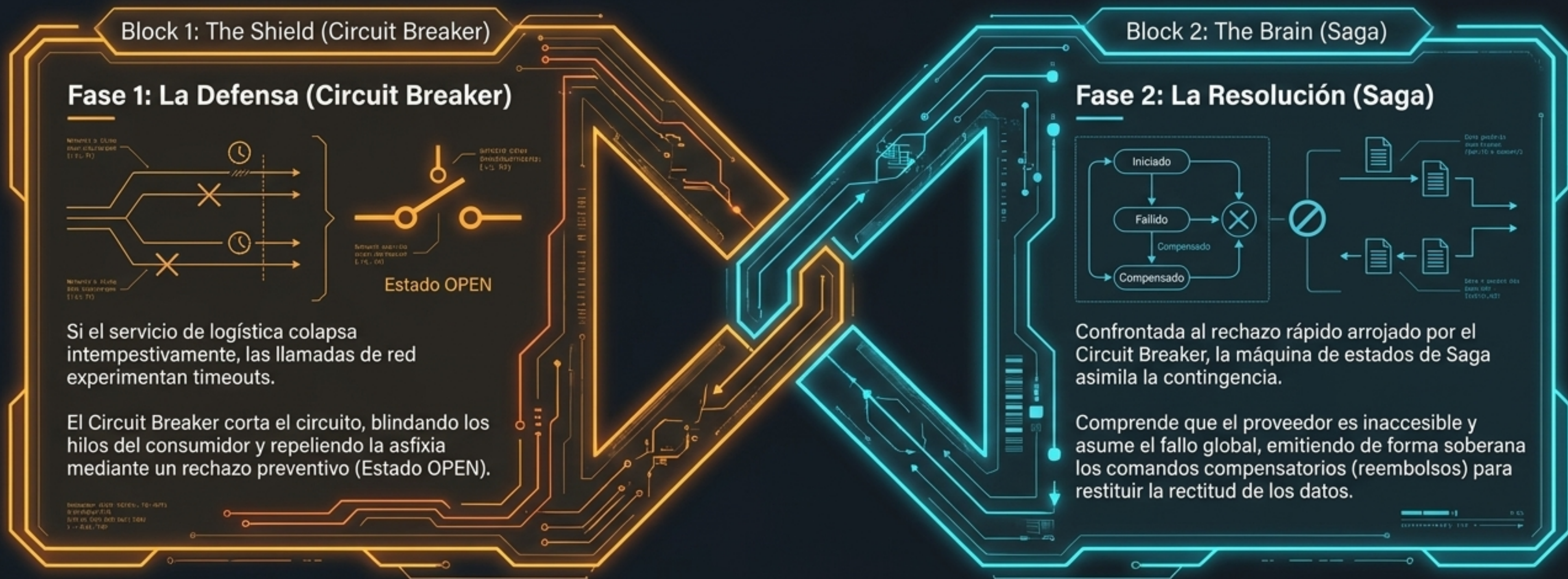
Paradigmas Ortogonales

Distinguiendo las responsabilidades en arquitecturas Cloud-Native.

Dimensión	Patrón Saga (Consistencia)	Circuit Breaker (Disponibilidad)
El Problema Base	Secuestro de recursos por bloqueos persistentes (2PC). 	Intermitencias foráneas e inanición de hilos. 
Foco de Resolución	Lógica funcional (Rechazos por negocio: sin stock). 	Congestión de red y latencia de hardware. 
Medio de Comunicación	Eventos Asíncronos (RabbitMQ / Message Brokers). 	Invocaciones Sincrónicas (HTTP / REST / gRPC). 
Estrategia de Fallo	Emite comandos de transacción compensatoria a bases de datos. 	Detiene la transmisión (Fail-Fast) y retorna un Fallback local. 
Ciclo de Estados	Orientado al negocio (Iniciado, Fallido, Compensado). 	Orientado a la red (Cerrado, Abierto, Medio-Abierto). 

Simbiosis Arquitectónica

La resiliencia definitiva nace de la colaboración de ambos patrones.



Insight Final: El Circuit Breaker apantalla las vías comunicativas de la infraestructura; la Saga ejerce la validación incuestionable del modelo de datos resultante.